Machine Learning Series — Volume 3

DEEP LEARNING

A Complete Beginner-to-Practitioner Guide

Covering: History · Perceptron · MLP · ANN Design · Layers · Activations
Computation Graph · Gradient Descent · Backpropagation · Optimizers · Keras
Classification & Regression with Neural Networks · Overfitting · Regularisation

Python / TensorFlow / Keras / sklearn Edition

00 What is Deep Learning?

The story, the intuition, and why it changed everything.

0.1 A Simple Definition

Deep Learning is a branch of machine learning that uses **artificial neural networks with many layers** to automatically learn complex patterns from raw data. The word **"deep"** refers to the depth of these layers — instead of one or two, a deep network might have 10, 50, or even hundreds of layers, each one learning progressively more abstract features.

The key breakthrough: instead of humans hand-crafting features (like "edge detection" or "roundness"), deep learning lets the network **discover features on its own** directly from raw pixels, audio waveforms, or text characters.

Everyday Analogy: Learning to Recognise a Dog

Traditional ML: Engineer manually codes features — "has 4 legs", "has fur", "has tail"
Then feeds these features into a classifier.

Deep Learning: Show the network millions of dog photos with no instructions.
Layer 1 learns: edges and lines
Layer 2 learns: eyes, noses, ears
Layer 3 learns: faces, paws
Layer 4 learns: "this whole arrangement = dog"
The network builds its OWN feature hierarchy automatically.

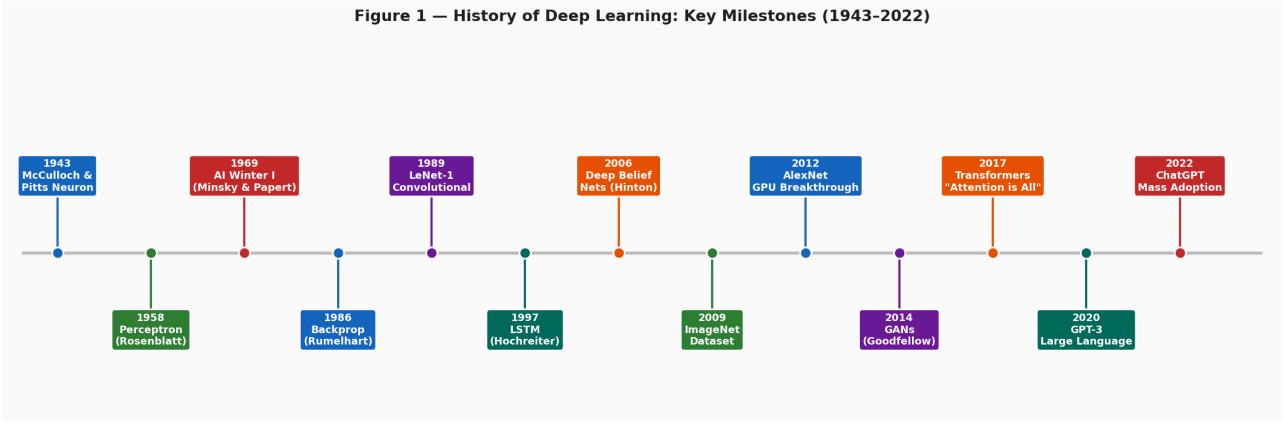


Figure 1 — Key milestones in the history of Deep Learning (1943–2022).

0.2 History — How We Got Here

The ideas behind neural networks date back to **1943**, when Warren McCulloch and Walter Pitts published the first mathematical model of a neuron. But the field experienced several **"AI Winters"** — periods of reduced funding and interest — before the modern explosion began around 2012.

Era	Key Development	Why It Mattered
1943	McCulloch-Pitts neuron	First mathematical model of a neuron
1958	Perceptron (Rosenblatt)	First trainable single-layer neural network
1969	Minsky & Papert critique	Showed perceptrons couldn't learn XOR → AI Winter 1
1986	Backpropagation (Rumelhart)	Made training multi-layer networks possible
1989	LeNet (LeCun)	First CNN for digit recognition
1997	LSTM (Hochreiter)	Solved vanishing gradient for sequences
2006	Deep Belief Nets (Hinton)	Showed deep networks could be pre-trained layer by layer
2012	AlexNet (GPU + ReLU + Dropout)	Won ImageNet — 10% gap. Started modern DL era.
2014	GANs (Goodfellow)	Generative adversarial networks
2017	Transformer ("Attention is All You Need")	Foundation of GPT, BERT, modern NLP
2022	ChatGPT	Mass public adoption of large language models

0.3 What is Deep Learning Best For?

Domain	Task	Why DL Wins Here
Computer Vision	Image classification, object detection, segmentation	Learns hierarchical spatial features from pixels automatically
Natural Language	Translation, summarisation, Q&A, generation	Captures long-range dependencies in text (Transformer)
Speech	Voice recognition, text-to-speech	Models temporal audio patterns at multiple timescales
Medicine	Disease detection from scans, drug discovery	Finds subtle patterns in high-dimensional medical images
Gaming / RL	Atari, Chess, Go, robotics control	Can learn policies directly from pixels + rewards
Tabular / Mixed	When data is large (>100K) and relationships complex	Can capture non-linear feature interactions
Generative	Image/text/music/video generation	GANs, VAEs, Diffusion models, LLMs

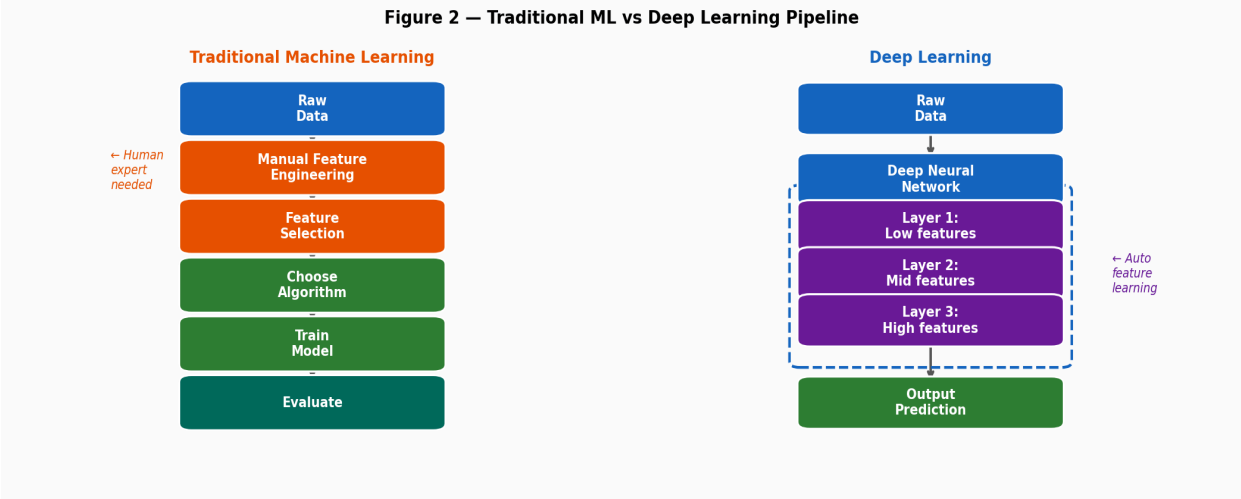


Figure 2 — Traditional ML (requires manual feature engineering) vs Deep Learning (learns features automatically).

0.4 When NOT to Use Deep Learning

Deep Learning is powerful but is **not always the best choice**. Consider traditional ML when:

- **You have < 10,000 samples** — deep networks need large amounts of data to generalise
- **Your data is tabular/structured** — Random Forest & XGBoost often outperform on small tabular data
- **Interpretability is critical** — "why did the model decide this?" is hard with deep networks
- **Compute budget is limited** — training a deep network requires significant GPU time
- **Your problem is already solved well** — if Linear Regression gives 95% accuracy, don't over-engineer

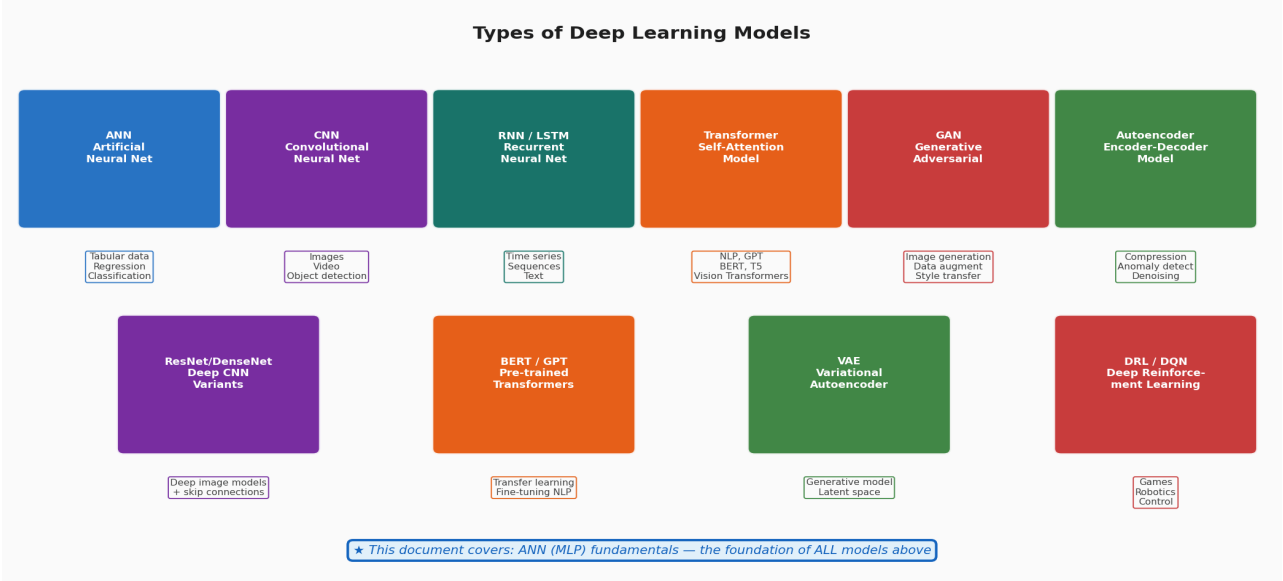


Figure 3 — Types of deep learning models. This guide covers the ANN/MLP foundation shared by all of them.

0.5 Types of Deep Learning Models

Model	Full Name	What It Processes	Key Use Cases
ANN / MLP	Artificial Neural Net / Multi-Layer Perceptron	Flat feature vectors (tabular)	Classification, Regression on structured data

CNN	Convolutional Neural Network	Grid data (images, video)	Image classification, object detection, face recognition
RNN / LSTM	Recurrent / Long Short-Term Memory	Sequences (text, time series, audio)	Language modelling, translation, forecasting
Transformer	Self-Attention Model	Any sequence (text, image, audio)	GPT, BERT, T5, Vision Transformers
GAN	Generative Adversarial Network	Images / data	Image generation, deepfakes, data augmentation
Autoencoder	Encoder-Decoder Network	Any data	Compression, anomaly detection, denoising
VAE	Variational Autoencoder	Any data	Generative models with controlled latent space
DRL	Deep Reinforcement Learning	States + actions	Games, robotics, autonomous driving

01 The Perceptron

The artificial neuron — the single building block of all neural networks.

1.1 Biological Inspiration

Every deep learning model — from a simple perceptron to GPT-4 — is built from one primitive unit: the **artificial neuron**, modelled after the biological neuron in your brain. Understanding this unit deeply means you understand the foundation of everything that follows.

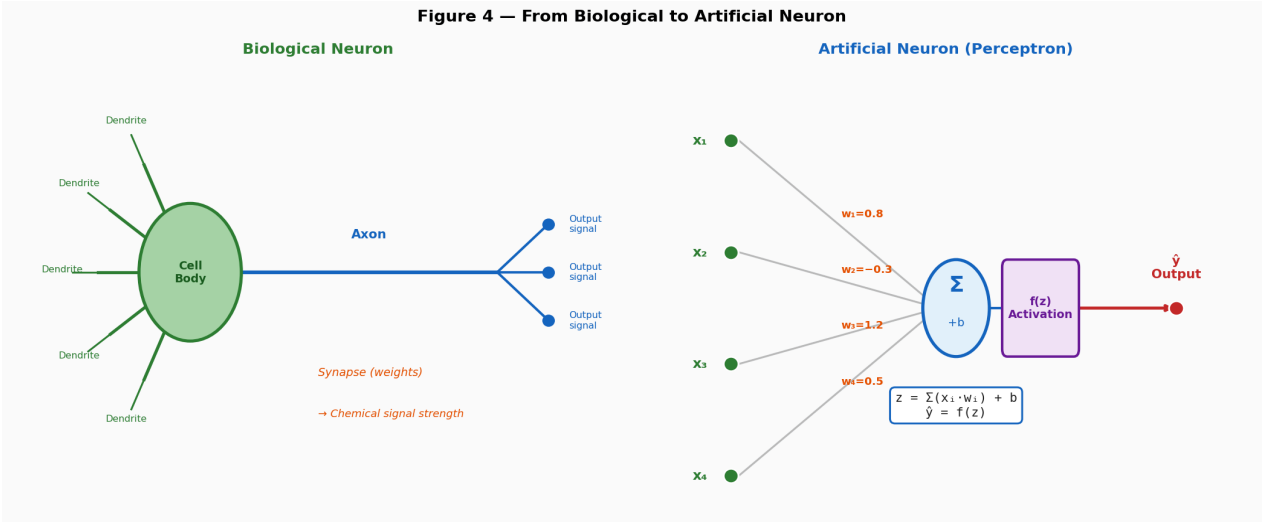


Figure 4 — Left: biological neuron with dendrites (inputs), cell body (computation), axon (output). Right: the artificial neuron mirrors this exactly.

1.2 The Single Neuron (Perceptron)

A single artificial neuron receives **multiple inputs**, multiplies each by a **weight**, adds a **bias**, and passes the result through an **activation function** to produce an output. This is the complete computation of one neuron:

The Neuron Equation — Step by Step

Step 1: Weighted Sum → $z = (x_1 \times w_1) + (x_2 \times w_2) + \ldots + (x_n \times w_n) + b$
$= \Sigma\,(x_i \times w_i) + b$

Step 2: Activation → $\hat{y} = f(z)$
Pass z through an activation function f

Example: x=[0.5, 0.8, 0.2], w=[0.9, -0.4, 0.6], b=0.1
z = (0.5×0.9) + (0.8×-0.4) + (0.2×0.6) + 0.1
z = 0.45 + (-0.32) + 0.12 + 0.10 = 0.35
ŷ = ReLU(0.35) = 0.35 (output of this neuron)

What are Weights?

Weights (w_1, w_2, ...) are the **learnable parameters** of the network. Each weight controls how much influence a particular input has on the output. A large positive weight means "this input strongly predicts the output". A negative weight means "when this input is high, the output decreases". The **training process** is entirely about finding the best values for all the weights.

What is the Bias?

The bias (b) is an extra learnable parameter added to every neuron. It shifts the activation function left or right, allowing the model to fit data that doesn't pass through the origin. Without bias, every neuron would output exactly 0 when all inputs are 0 — but in many real problems, the answer when all inputs are 0 is **not** zero. Bias fixes this.

Component	Symbol	Role	Initialisation
Input	x_i	The features fed into the neuron. Fixed — not learned.	From dataset
Weight	w_i	Controls how much each input contributes. Learned.	Random small values (~N(0,0.01))
Bias	b	Shifts the output. Learned parameter.	Usually 0 at start
Weighted sum	z	Linear combination: $z = \Sigma(x_i \cdot w_i) + b$	Computed each forward pass
Activation	f(z)	Applies non-linearity. Transforms z into output.	Chosen by designer
Output	$\hat{y}$	Result of f(z). Fed to next layer or used as prediction.	Computed each forward pass

1.3 A Perceptron Cannot Learn XOR — Why We Need Layers

The original Perceptron (1958) could only solve **linearly separable** problems — problems where you can draw a single straight line to separate the classes. In 1969, Minsky and Papert proved that a single-layer perceptron **cannot learn the XOR function** (exclusive OR). This discovery caused the first AI Winter. The solution, discovered in 1986: **stack multiple layers** — the Multi-Layer Perceptron.

Input A	Input B	XOR Output	Linear Separable?
0	0	0	—
0	1	1	—
1	0	1	—
1	1	0	No — single line cannot separate this

02 Layers: The Building Blocks

Input, hidden, and output layers — every detail explained.

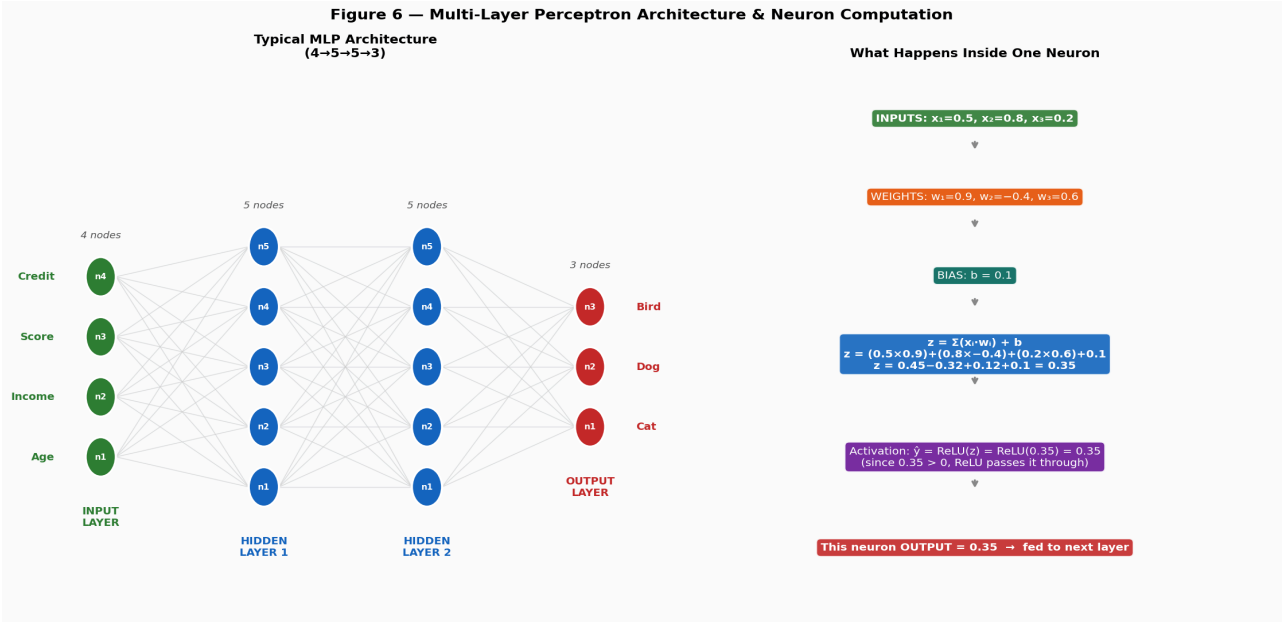


Figure 6 — Left: MLP with 4 layers. Right: what happens inside one neuron — weights, bias, activation, and output.

2.1 The Input Layer

The **input layer** is the first layer and is simply a **pass-through for your features**. It does **no computation** — it just receives the data and passes it to the first hidden layer. The number of input neurons equals the number of features in your data.

Dataset	Input Features	Input Layer Size	Example input vector
Iris	4 measurements	4 neurons	[5.1, 3.5, 1.4, 0.2] (sepal L/W, petal L/W)
Breast Cancer	30 cell features	30 neurons	[17.99, 10.38, ..., 0.07] (30 measurements)
MNIST digit	28×28 image	784 neurons	Flatten 28×28 pixels → [0.0, 0.0, ..., 1.0]
Tabular data	5 columns	5 neurons	[age, salary, score, years_exp, dept_code]

How to Feed Data into the Input Layer

1. SCALE your features — always. Use StandardScaler (mean=0, std=1) or MinMaxScaler (0-1)
   Without scaling, large-range features dominate and training is unstable.

2. FLATTEN images — a 28×28 image becomes a 784-element 1D vector for ANN.
(CNN handles 2D spatial data differently — no flattening needed)

3. ENCODE categories — turn "cat/dog/bird" into [0,1,2] then one-hot [1,0,0]/[0,1,0]/[0,0,1]
Or use Embedding layers for many categories.

4. HANDLE missing values — neural networks cannot handle NaN.
Impute (fill mean/median) or use a masking strategy.

2.2 Hidden Layers

Hidden layers are the computational heart of the network. Each hidden layer receives the outputs of the previous layer, applies a **linear transformation (weights + bias)**, then a **non-linear activation function**, and passes its outputs forward. The word "hidden" just means the values are not directly visible in input or output — they are internal representations.

How Many Hidden Layers?

Depth	Layer Count	Typical Use Case
Shallow	1 hidden layer	Simple problems, small datasets, quick baseline
Moderate	2–3 hidden layers	Most tabular classification/regression problems
Deep	4–8 hidden layers	Complex structured data, audio, medium-sized NLP
Very Deep	8–50+ hidden layers	Images (ResNet), NLP (BERT), large-scale tasks

Practical Rules for Choosing Hidden Layers

Rule 1: Start with 1–2 hidden layers. Add more only if underfitting.
Rule 2: More hidden layers = more expressive, but also more risk of overfitting.
Rule 3: Universal Approximation Theorem says 1 hidden layer can approximate
ANY function — but it may need an impractically large number of neurons.
Rule 4: Depth (more layers) often works better than width (more neurons per layer)
for learning complex hierarchical patterns.

2.3 How Many Neurons Per Hidden Layer?

The number of neurons in a hidden layer is called the **layer width** or **units**. There is no single correct answer, but the following guidelines work well for most problems:

Problem Size	Recommended Width	Example Architecture
Very small (< 1K samples)	16 – 64 neurons per layer	Input → 32 → 16 → Output
Small (1K – 10K samples)	64 – 128 neurons per layer	Input → 128 → 64 → Output

Medium (10K – 100K samples)	128 – 512 neurons per layer	Input → 256 → 128 → 64 → Output
Large (> 100K samples)	512 – 2048 neurons per layer	Input → 1024 → 512 → 256 → Output

Common Architecture Patterns

PYRAMID (decreasing): Input → 256 → 128 → 64 → Output
Best for: general classification. Compresses information gradually.

CONSTANT WIDTH: Input → 256 → 256 → 256 → Output
Best for: when you don't want to lose information too early.

BOTTLENECK: Input → 128 → 32 → 128 → Output
Best for: autoencoders, representation learning, compression tasks.

EXPANDING: Input → 32 → 64 → 128 → Output
Best for: generative models, upsampling tasks.

2.4 The Output Layer

The **output layer** is the final layer. Its size and activation function depend entirely on the task. This is one of the most important design decisions:

Task	Output Neurons	Activation	Loss Function	Example
Binary Classification	1	Sigmoid → output 0–1	Binary Cross-Entropy	Is email spam? (0=no, 1=yes)
Multi-Class Classification	K (# classes)	Softmax → K probabilities summing to 1	Categorical Cross-Entropy	Which digit 0-9? (K=10)
Multi-Label Classification	K (# labels)	Sigmoid on each (independent)	Binary Cross-Entropy	Image tags (cat AND dog?)
Regression (single value)	1	Linear (no activation)	MSE or MAE	Predict house price
Regression (multi-output)	K values	Linear	MSE	Predict x,y,z coordinates

2.5 Multiple Output Nodes — Classification Detail

When predicting **multiple classes** (e.g., cat, dog, bird = 3 classes), the output layer has **one node per class**. The **Softmax activation** ensures all output values are positive and sum to exactly 1.0 — making them interpretable as **probabilities**.

Softmax Output — Worked Example

Input to output layer (raw logits): z = [2.1, 0.8, -0.5]

Softmax step 1: exp(z) = [e^2.1, e^0.8, e^-0.5]
= [8.17, 2.23, 0.61]

Softmax step 2: sum = 8.17 + 2.23 + 0.61 = 11.01

Softmax step 3: divide each by sum:
[8.17/11.01, 2.23/11.01, 0.61/11.01]
= [0.742, 0.203, 0.055]

Output: Cat=74.2%, Dog=20.3%, Bird=5.5%
Prediction: Cat (highest probability)
All probabilities sum to: 0.742 + 0.203 + 0.055 = 1.000 ✓

03 Activation Functions

The non-linearity that gives neural networks their power.

Without activation functions, stacking multiple layers would be mathematically equivalent to a **single linear layer** — no matter how many layers you added. Activation functions introduce **non-linearity**, allowing the network to learn complex, curved decision boundaries and represent any function.

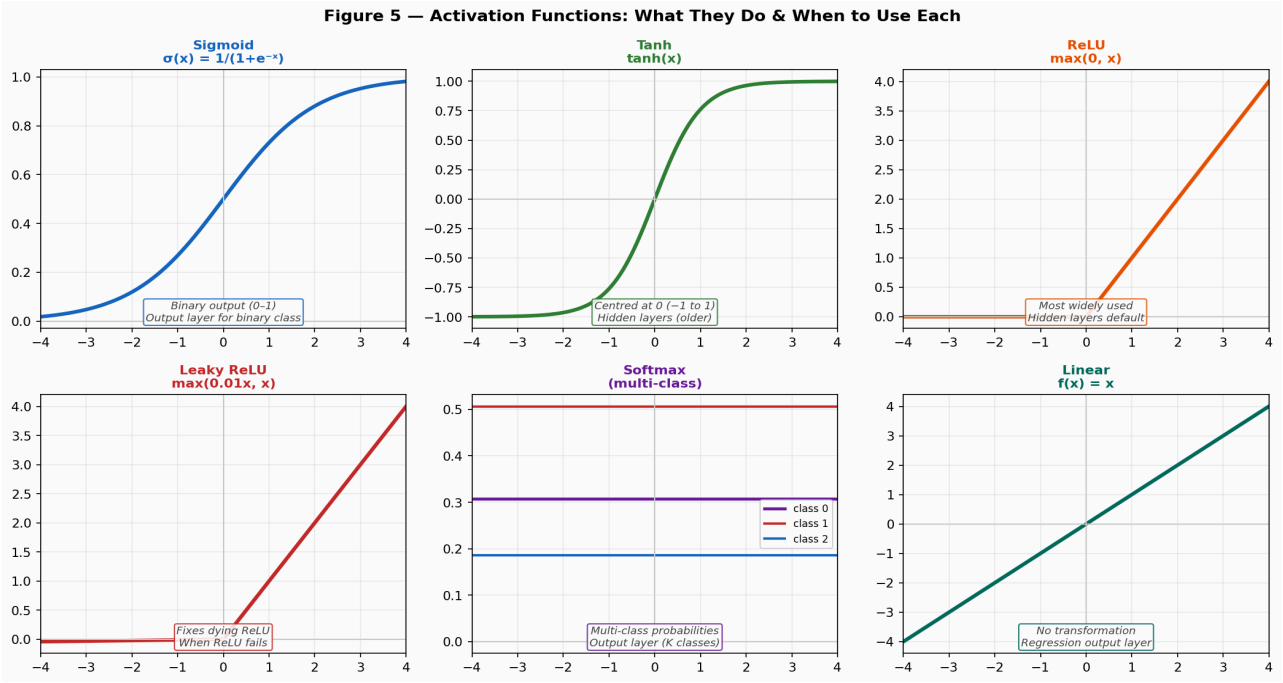


Figure 5 — Six activation functions: when to use each, their mathematical form, and typical application.

3.1 Sigmoid

Formula: `σ(x) = 1 / (1 + e^(-x))`

Output range: 0 to 1. Squashes any input to a probability-like value. Use **only in the output layer for binary classification**. **Never in hidden layers** — the sigmoid "saturates" (flat slope) at both ends, causing the **vanishing gradient problem** which makes deep networks stop learning.

3.2 Tanh (Hyperbolic Tangent)

Formula: `tanh(x) = (e^x - e^(-x)) / (e^x + e^(-x))`

Output range: −1 to 1. Zero-centred (unlike sigmoid). Better than sigmoid for hidden layers in older architectures. Still suffers from vanishing gradients in very deep networks. You will see it used in **RNNs and LSTMs**.

3.3 ReLU — The Default Choice

Formula: `ReLU(x) = max(0, x)`

Use ReLU in almost all hidden layers by default. It is computationally cheap (just a max), does not saturate for positive values, and dramatically accelerated deep network training. The 2012 AlexNet breakthrough was partly

due to ReLU replacing sigmoid/tanh. **Downside**: "Dying ReLU" — if a neuron gets a large negative input repeatedly, its gradient becomes 0 and the neuron stops learning ("dies").

3.4 Leaky ReLU & ELU

Leaky ReLU formula: `f(x) = x if x > 0 else α·x (α=0.01)`

Fixes the dying ReLU problem by allowing a small negative slope. **Use when you observe dying neurons** (many neurons outputting 0). ELU (Exponential Linear Unit) is smoother and often slightly better than Leaky ReLU in practice.

3.5 Softmax

Use ONLY in the output layer for multi-class classification. Converts raw logits into probabilities that sum to 1. Never use in hidden layers.

3.6 Linear (No Activation)

Use in the output layer for regression. No transformation is applied — the raw weighted sum z is the output. This allows any continuous value (negative or positive) to be predicted, which is what regression requires.

Activation	Output Range	Use In	Do NOT Use For
Sigmoid	0 to 1	Output: binary classification	Hidden layers — causes vanishing gradient
Tanh	−1 to 1	RNN/LSTM hidden states	Deep MLP hidden layers
ReLU	0 to ∞	Hidden layers — DEFAULT CHOICE	Output layer (loses negative values)
Leaky ReLU	−∞ to ∞	Hidden layers — when ReLU fails	Output layer
Softmax	0 to 1 (sums to 1)	Output: multi-class classification	Hidden layers, binary output
Linear	−∞ to ∞	Output: regression	Hidden layers (no non-linearity)
GELU / Swish	−0.2 to ∞	Transformers, modern architectures	Beginners — use ReLU first

04 Computation Graph & Backpropagation

How neural networks actually learn — the mathematics made visual.

4.1 What is a Computation Graph?

A **computation graph** is a visual representation of all the mathematical operations in your neural network, drawn as a directed graph where each **node = one operation** (multiply, add, activation) and each **edge = data flowing** between operations. Frameworks like TensorFlow and PyTorch build this graph automatically so that gradients can be computed efficiently via **automatic differentiation**.

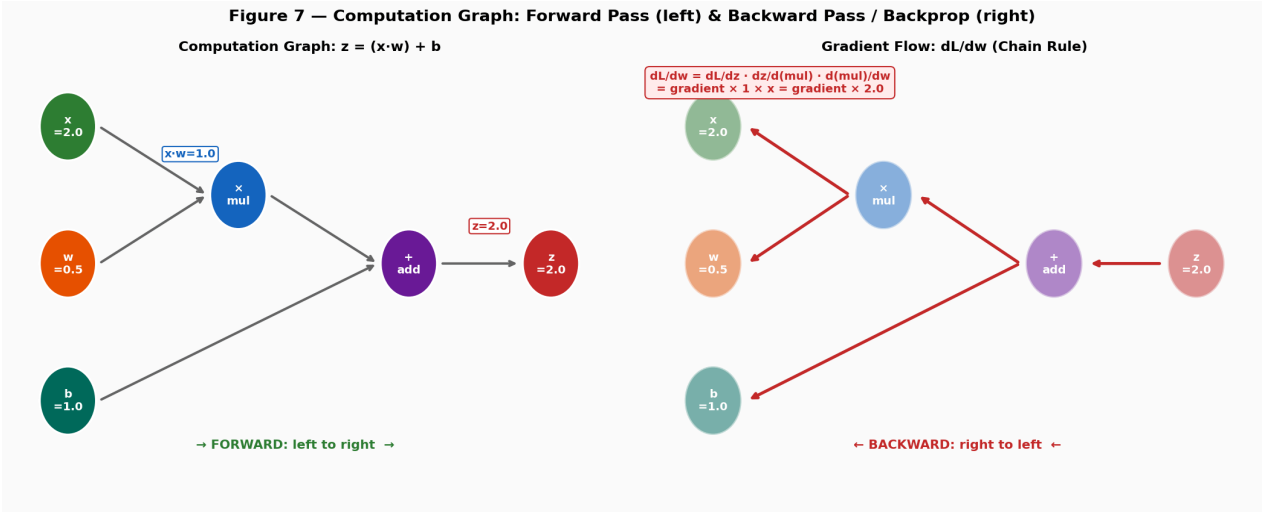


Figure 7 — Computation graph for z=x·w+b. Left: forward pass (left to right). Right: backward pass — gradients flow right to left via chain rule.

4.2 The Forward Pass

The forward pass is the step where input data flows **from left to right** through the network, each layer performing its computation, until we get the prediction ŷ. Then we compute the **loss** — how far off we were from the true answer.

Forward Pass — Step by Step for One Neuron

Input: x = [2.0, 1.5]
Weights: w = [0.5, -0.3], Bias: b = 0.1

1. Weighted sum: z = (2.0×0.5) + (1.5×-0.3) + 0.1
   z = 1.0 - 0.45 + 0.1 = 0.65

2. Activation: a = ReLU(0.65) = 0.65

3. Output layer: ŷ = sigmoid(a·w_out + b_out) ⟶ some probability

4. Loss: L = -[y·log(ŷ) + (1-y)·log(1-ŷ)] (binary cross-entropy)

= measures how wrong the prediction is

4.3 The Backward Pass (Backpropagation)

Backpropagation is the algorithm for computing **gradients** — how much the loss changes when each weight changes. It works by applying the **chain rule of calculus** backwards through the computation graph: starting from the loss, it propagates gradient information from the output back to every single weight in the network.

The Chain Rule Made Simple

If $L = f(g(x))$, then $dL/dx = dL/dg \times dg/dx$

In a neural network:
$dL/dw_1 = dL/d\hat{y} \times d\hat{y}/da \times da/dz \times dz/dw_1$

Read this as:
"How does loss change if we tweak ŷ?" × "How does ŷ change if we tweak a?"
× "How does a change if we tweak z?" × "How does z change if we tweak w_1?"

Each factor is easy to compute. Multiplied together = gradient of w_1.
Frameworks (TensorFlow, PyTorch) compute this automatically for you.

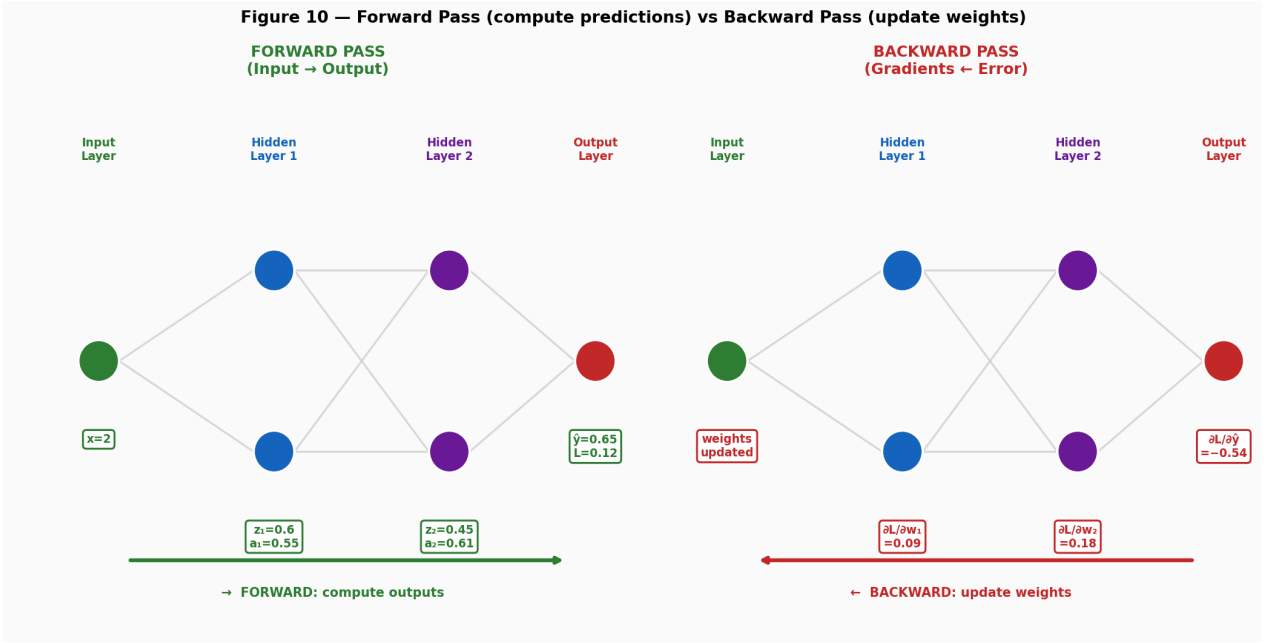


Figure 10 — Forward pass (left): data flows input to output, values computed. Backward pass (right): gradients flow output to input, weights updated.

4.4 Weight Update Rule

After backpropagation computes all gradients, we **update every weight** using:

Gradient Descent Weight Update

$w_{new} = w_{old} - \alpha \times \partial L/\partial w$

w_{old} = current weight value
α = learning rate (e.g., 0.001) — how big a step to take
$\partial L/\partial w$ = gradient — direction to move to reduce loss
minus sign = we SUBTRACT gradient (move opposite to gradient = downhill on loss surface)

Example: $w = 0.5$, gradient = 0.3, $\alpha = 0.1$
$w_{new} = 0.5 - 0.1 \times 0.3 = 0.5 - 0.03 = 0.47$
The weight moved slightly in the direction that reduces loss.

05 Gradient Descent & Optimizers

How the network learns — finding the minimum of the loss function.

Figure 8 — Loss Functions & Gradient Descent Visualised

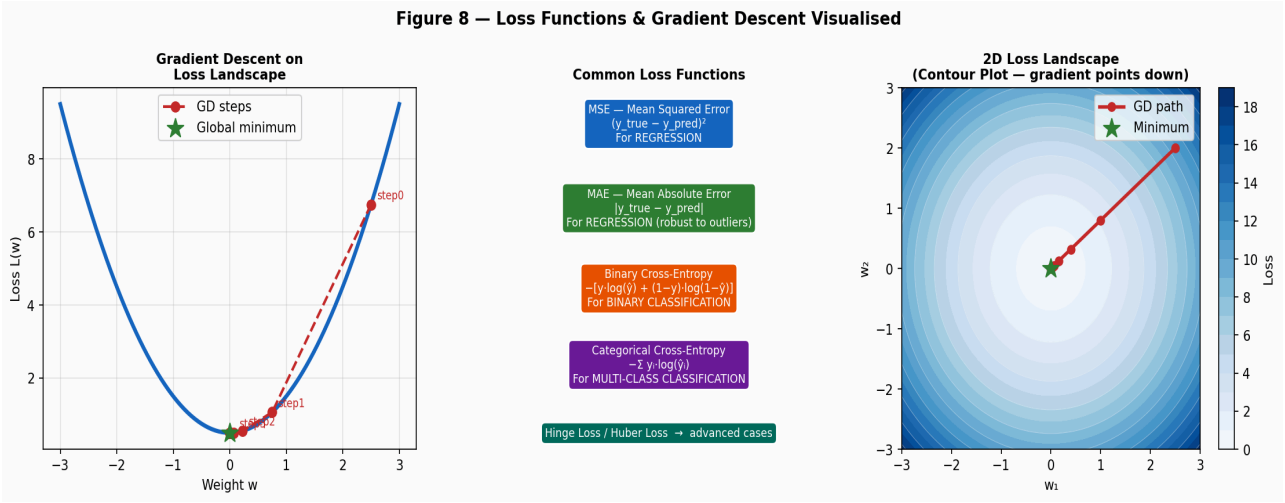


Figure 8 — Left: GD steps descending the loss curve. Centre: loss function types. Right: 2D loss landscape with gradient path.

5.1 The Loss Function

The **loss function** (also called cost function) measures **how wrong the model is** on a given batch of data. It takes the true labels y and predicted labels ŷ, and returns a single number. Our goal in training is to **minimise this number** by adjusting the weights.

Loss Function	Formula (simplified)	Use For	Lower Bound
MSE	$(1/n)\ \Sigma\ (y - \hat{y})^2$	Regression	0 (perfect)
MAE	$(1/n)\ \Sigma\ \lvert y - \hat{y}\rvert$	Regression (robust)	0 (perfect)
Binary Cross-Entropy	$-[y\cdot\log(\hat{y})+(1-y)\cdot\log(1-\hat{y})]$	Binary classification	0 (perfect)
Categorical Cross-Entropy	$-\Sigma\ y_i\cdot\log(\hat{y}_i)$	Multi-class classification	0 (perfect)
Huber Loss	MSE when small, MAE when large	Regression + outliers	0 (perfect)

5.2 Gradient Descent Variants

Gradient Descent is the optimisation algorithm. But how much data do we use to compute each gradient update? This choice defines **three variants**:

Variant	Data per Update	Speed	Noise Level	Best For
Batch GD	All training data	Slow (one update per epoch)	Smooth path	Small datasets,

				exact gradients
Stochastic GD (SGD)	One sample at a time	Fast (n updates per epoch)	Very noisy	Online learning, huge datasets
Mini-batch GD	32–256 samples	Fast + stable	Moderate	DEFAULT — almost always use this

Mini-batch Size Guidelines

- 32 — default safe choice for most problems
- 64 — good balance of speed and accuracy, works on most GPUs
- 128 — faster training if you have enough GPU memory
- 256+ — use for very large datasets, need more GPU memory
- Rule: power of 2 (32, 64, 128, 256) to maximise GPU efficiency

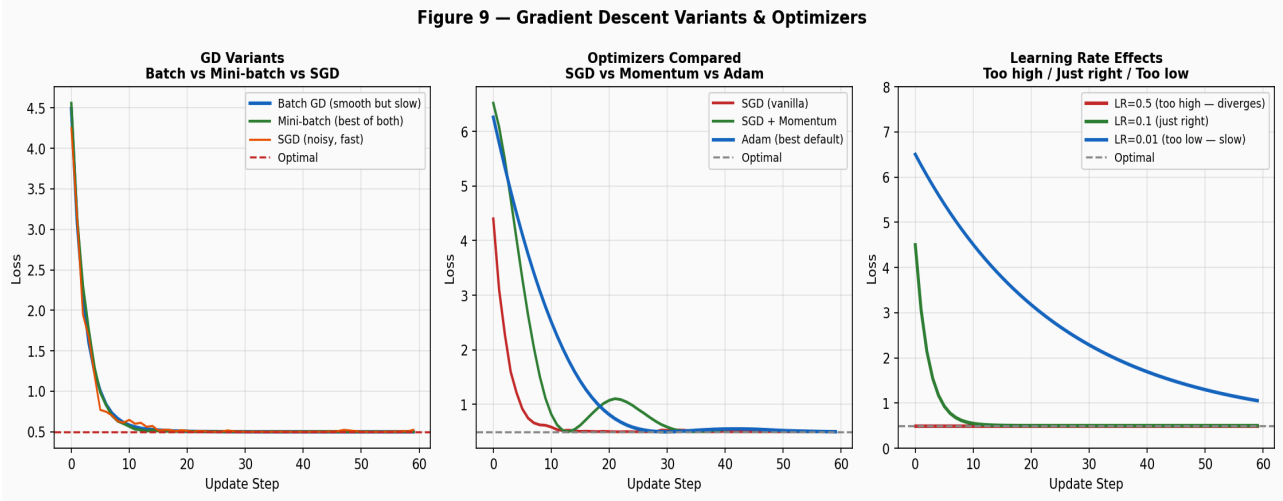


Figure 9 — Left: Batch vs Mini-batch vs SGD convergence. Centre: SGD vs Momentum vs Adam. Right: Effect of learning rate.

5.3 Optimizers

An **optimizer** is an algorithm that improves upon vanilla gradient descent by adapting the step size or using momentum to converge faster and more reliably.

Optimizer	Key Idea	When to Use
SGD (vanilla)	Plain gradient descent with fixed learning rate	Simple problems, well-tuned LR, research baselines
SGD + Momentum	Accumulate velocity in gradient direction (like a ball rolling)	Faster than SGD; good for convex problems

RMSprop	Divide LR by moving avg of squared gradients (adaptive per-param)	RNNs, non-stationary data
Adam	Momentum + adaptive LR (combines Momentum + RMSprop)	DEFAULT for almost all deep learning
AdamW	Adam + weight decay (better regularisation)	Transformers, large models, fine-tuning
Nadam	Adam + Nesterov momentum (look-ahead gradient)	Slight improvement over Adam in practice

The Learning Rate — The Most Important Hyperparameter

Too HIGH: loss oscillates wildly or diverges (explodes to infinity)
Too LOW: training is extremely slow; may get stuck before converging
Just right: loss decreases steadily and converges to a good minimum

Default starting values:
Adam: $\alpha = 0.001$ (almost always the best first try)
SGD: $\alpha = 0.01$ (often needs tuning)

Pro tip: Use a Learning Rate Scheduler to reduce LR during training:
ReduceLROnPlateau: halve LR when val_loss stops improving
CosineDecay: smoothly decrease from max to near-zero LR

06 Training, Overfitting & Regularisation

How to train well and prevent your model from memorising the data.

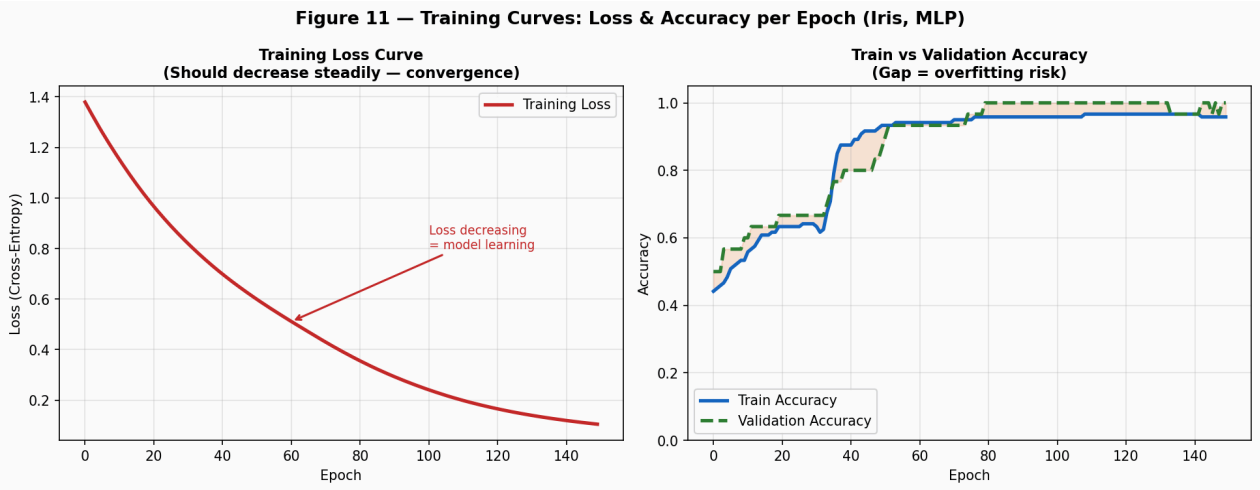


Figure 11 — Left: training loss decreasing ⟶ model learning. Right: training vs validation accuracy. Gap between curves = overfitting.

6.1 The Training Loop

Training a neural network follows the same cycle every epoch:

1. **Forward pass:** feed batch through network, compute predictions ŷ
2. **Compute loss:** measure error between ŷ and true y
3. **Backward pass:** backpropagation — compute all gradients
4. **Update weights:** optimizer applies gradients to adjust all weights
5. **Repeat for all batches** in the dataset (one epoch done)
6. **Evaluate on validation set** at end of each epoch
7. **Repeat for N epochs** until loss stops improving

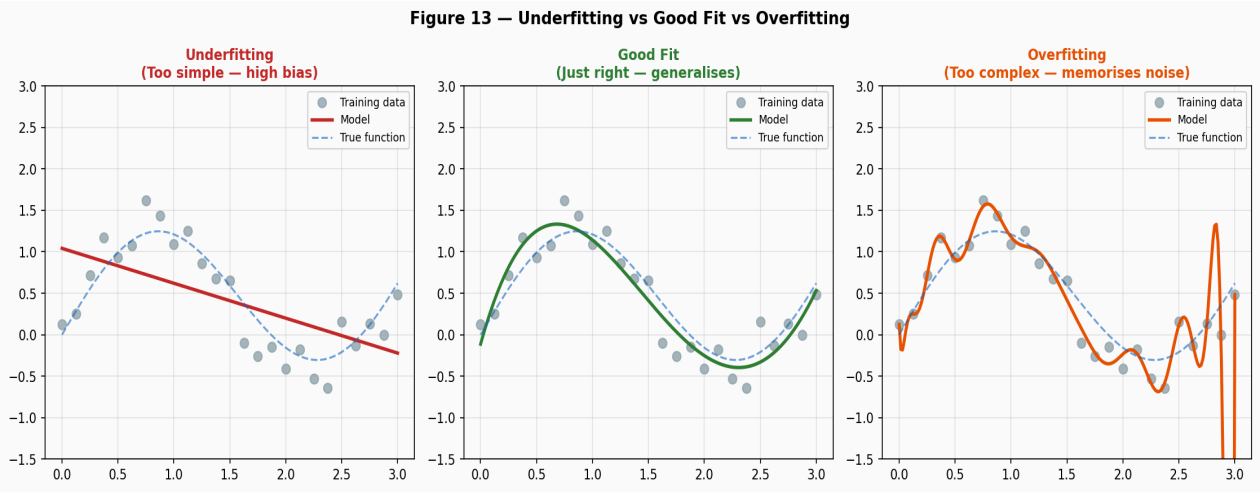


Figure 13 — Underfitting (model too simple), Good fit (generalises), Overfitting (memorises training noise).

6.2 Underfitting vs Overfitting

Problem	Training Loss	Validation Loss	Cause	Fix
Underfitting	High	High	Model too simple / not enough capacity	More layers, more neurons, train longer
Good fit	Low	Low (similar)	Model right for the data	Keep this!
Overfitting	Low	High	Model memorised training noise	Regularisation, more data, dropout

6.3 Regularisation Techniques

Regularisation is any technique that helps the model generalise better to unseen data by preventing overfitting:

Technique	What It Does	Keras Implementation
L2 / Weight Decay	Adds penalty for large weights to loss: $+\lambda\Sigma w^2$	kernel_regularizer=l2(0.01) in Dense()
L1 Regularisation	Sparse weights: pushes some exactly to 0	kernel_regularizer=l1(0.01) in Dense()
Dropout	Randomly sets fraction of neurons to 0 during training	Dropout(0.3) layer after Dense()
Batch Normalisation	Normalises layer inputs; stabilises training; slight regularisation	BatchNormalization() after Dense()
Early Stopping	Stop training when val_loss stops improving	EarlyStopping(patience=10)
Data Augmentation	Artificially increase training data variety (rotation, flip, etc.)	ImageDataGenerator, albumentations

Dropout — The Most Widely Used Regulariser

During TRAINING: each neuron is randomly "dropped" (set to 0)
with probability p (e.g., p=0.3 means 30% of neurons are dropped each step)

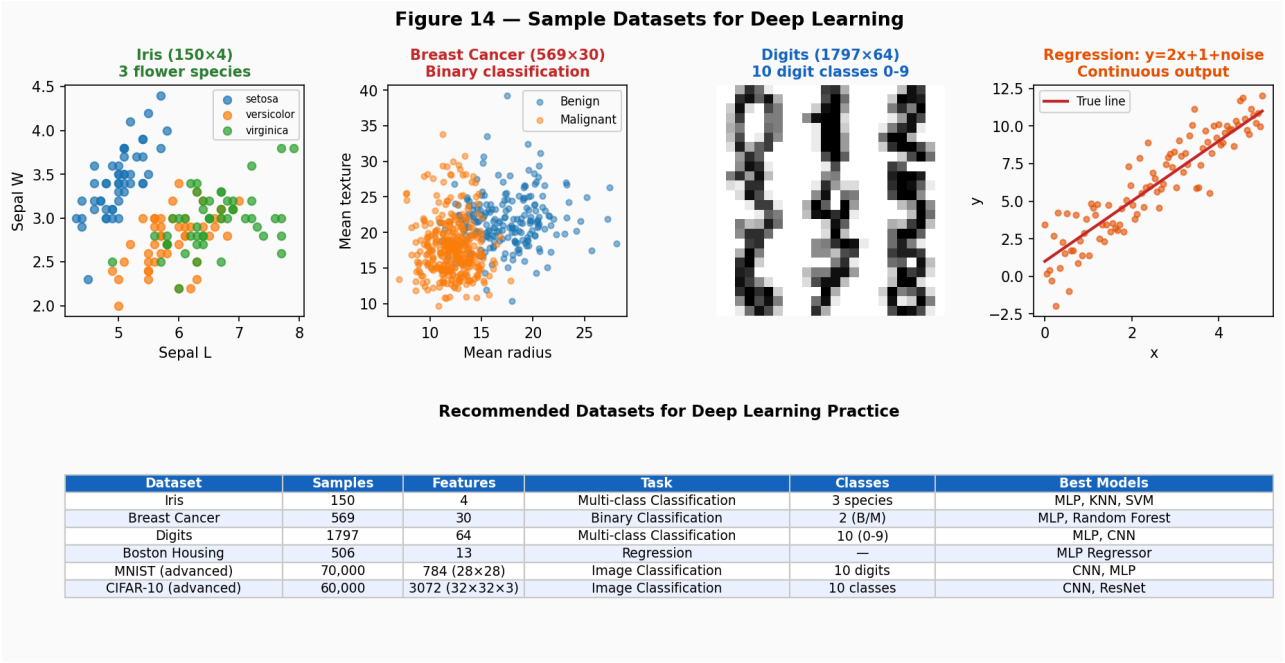
During INFERENCE/TEST: all neurons active, but outputs scaled by (1-p)

Effect: forces the network to learn REDUNDANT representations.
No single neuron can become the only neuron that knows something.
Makes the network more robust — like training an ensemble of models.

Typical dropout rates: 0.2-0.3 for hidden layers; 0.5 for very large layers

07 Datasets for Deep Learning Practice

Which dataset to pick for each type of model and task.



Dataset	Samples	Features	Task	Classes	Best Models
Iris	150	4	Multi-class Classification	3 species	MLP, KNN, SVM
Breast Cancer	569	30	Binary Classification	2 (B/M)	MLP, Random Forest
Digits	1797	64	Multi-class Classification	10 (0-9)	MLP, CNN
Boston Housing	506	13	Regression	—	MLP Regressor
MNIST (advanced)	70,000	784 (28×28)	Image Classification	10 digits	CNN, MLP
CIFAR-10 (advanced)	60,000	3072 (32×32×3)	Image Classification	10 classes	CNN, ResNet

Figure 14 — Common practice datasets: Iris, Breast Cancer, Digits, and a synthetic regression dataset. Bottom: full comparison table.

7.1 Beginner Datasets (sklearn built-in)

These datasets load with a single line of Python and require no downloading. Perfect for first experiments with neural networks:

Dataset	Load Command	Size	Task	Recommended Model
Iris	load_iris()	150 × 4	3-class classification	MLP: Input(4)→32→16→Output(3, softmax)
Breast Cancer	load_breast_cancer()	569 × 30	Binary classification	MLP: Input(30)→64→32→Output(1, sigmoid)
Wine	load_wine()	178 × 13	3-class classification	MLP: Input(13)→32→Output(3, softmax)
Digits	load_digits()	1797 × 64	10-class classification	MLP: Input(64)→128→64→Output(10, softmax)
Diabetes	load_diabetes()	442 × 10	Regression	MLP: Input(10)→64→32→Output(1, linear)
Boston Housing	fetch_california_housing()	20640 × 8	Regression	MLP: Input(8)→64→32→Output(1, linear)

7.2 Intermediate & Advanced Datasets

Dataset	Size	Task	Best Architecture	Notes
MNIST	70,000 × 784	Digit classification (0-9)	CNN or MLP	Classic benchmark; very high accuracy achievable
Fashion MNIST	70,000 × 784	Clothing classification (10 class)	CNN	Harder than MNIST; same format
CIFAR-10	60,000 × 32×32×3	Object classification (10 class)	CNN (ResNet)	Colour images; needs CNN, not MLP
IMDB Reviews	50,000 texts	Sentiment classification	LSTM, Transformer	Text — needs embedding + sequence model
Reuters News	11,228 texts	Multi-class text classification	LSTM, MLP on TF-IDF	46 categories
CIFAR-100	60,000 × 32×32×3	100-class classification	Deep CNN / ResNet	Harder version of CIFAR-10

Which Dataset for Which Task?

Learning ANN/MLP for the first time: → Start with Iris or Breast Cancer
Learning multi-class output: → Iris (3 classes) or Digits (10 classes)
Learning regression: → Diabetes or California Housing
Learning CNNs (next step after MLP): → MNIST, then CIFAR-10
Learning sequence models (RNN/LSTM): → IMDB Reviews
Learning Transformers / NLP: → IMDB, Reuters, or HuggingFace datasets

08 Designing ANNs with Keras

Full working code for classification and regression — explained line by line.

8.1 Keras Overview

Keras is a **high-level deep learning API** built on top of TensorFlow. It lets you design, train, and evaluate neural networks with clean, readable code. You describe the architecture layer by layer, compile with your chosen loss and optimizer, then call fit(). TensorFlow handles all the backpropagation automatically.

Framework	Level	Best For
Keras (tf.keras)	High-level API	Beginners, fast prototyping, standard architectures
TensorFlow 2	Mid-level	Production, custom training loops, deployment
PyTorch	Low-level	Research, flexible custom models, academia
sklearn MLPClassifier	Very high-level (limited)	Quick baselines without GPU, small datasets

8.2 Binary Classification — Breast Cancer (Keras)

• • • python

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# ── 1. Load and prepare data ──────────────────────────────
data   = load_breast_cancer()
X, y   = data.data, data.target          # 569 samples, 30 features, binary
label

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# ── ALWAYS SCALE ──────────────────────────────────────────
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)    # fit on train only!
X_test  = scaler.transform(X_test)         # transform test with train stats

# ── 2. Build the model ────────────────────────────────────
model = keras.Sequential([
    layers.Input(shape=(30,)),              # 30 input features

    layers.Dense(64, activation='relu'),    # Hidden layer 1: 64 neurons
    layers.BatchNormalization(),            # Stabilise training
    layers.Dropout(0.3),                   # 30% dropout = regularisation

    layers.Dense(32, activation='relu'),    # Hidden layer 2: 32 neurons
    layers.Dropout(0.2),
```

```
    layers.Dense(1, activation='sigmoid') # Output: 1 neuron, sigmoid → 0-1
])

model.summary()                           # Shows layers, params count

# ── 3. Compile ──────────────────────────────────────────────
model.compile(
    optimizer = keras.optimizers.Adam(learning_rate=0.001),
    loss      = 'binary_crossentropy',    # binary classification loss
    metrics   = ['accuracy']
)

# ── 4. Train ────────────────────────────────────────────────
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=15, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs          = 200,
    batch_size      = 32,
    validation_split= 0.15,   # 15% of training data used for validation
    callbacks       = [early_stop],
    verbose         = 1
)

# ── 5. Evaluate ─────────────────────────────────────────────
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc:.4f}")

# ── 6. Predictions ──────────────────────────────────────────
y_pred_prob = model.predict(X_test)       # probabilities 0-1
y_pred      = (y_pred_prob > 0.5).astype(int).flatten()

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred,
target_names=['Malignant','Benign']))
```

8.3 Multi-Class Classification — Iris (Keras)

• • • python

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

iris          = load_iris()
X, y          = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
X_train       = StandardScaler().fit_transform(X_train)
# (fit scaler on train, then use same scaler for test)

model = keras.Sequential([
    layers.Input(shape=(4,)),              # 4 features

    layers.Dense(32, activation='relu'),
```

```
    layers.Dense(16, activation='relu'),

    layers.Dense(3, activation='softmax') # 3 output neurons = 3 classes
    # Softmax → outputs [P(setosa), P(versicolor), P(virginica)]
    # All three probabilities sum to 1.0
])

model.compile(
    optimizer = 'adam',
    loss      = 'sparse_categorical_crossentropy',  # use when y is integers
0,1,2
    # Use 'categorical_crossentropy' if y is one-hot encoded
    metrics   = ['accuracy']
)

history = model.fit(X_train, y_train, epochs=100, batch_size=16,
                    validation_split=0.2, verbose=0)

print(f"Test acc: {model.evaluate(X_test, y_test, verbose=0)[1]:.4f}")

# Get probabilities for each class:
probs = model.predict(X_test[:3])
for i, p in enumerate(probs):
    pred_class = iris.target_names[np.argmax(p)]
    print(f"Sample {i+1}: {p.round(3)} → predicted: {pred_class}")
```

8.4 Regression — California Housing (Keras)

• • • python

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

housing = fetch_california_housing()
X, y    = housing.data, housing.target   # 20640 samples, 8 features, price

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = keras.Sequential([
    layers.Input(shape=(8,)),             # 8 input features

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(32, activation='relu'),

    layers.Dense(1)                       # 1 output, NO activation =
regression
    # Output is a raw continuous value (house price in $100K)
])
```

```
model.compile(
    optimizer = keras.optimizers.Adam(0.001),
    loss      = 'mse',                    # Mean Squared Error for
regression
    metrics   = ['mae']                   # Mean Absolute Error for
monitoring
)

model.fit(X_train, y_train, epochs=100, batch_size=64,
          validation_split=0.15, verbose=0)

y_pred = model.predict(X_test).flatten()
print(f"R² Score  : {r2_score(y_test, y_pred):.4f}")
print(f"RMSE      : {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
```

09 sklearn MLPClassifier & MLPRegressor

Train neural networks without TensorFlow — perfect for learning.

If you don't have TensorFlow installed, or want a quick baseline, sklearn includes **MLPClassifier** and **MLPRegressor**. These implement a fully-connected feedforward neural network with backpropagation. They are less flexible than Keras but excellent for learning and for tabular data benchmarks.

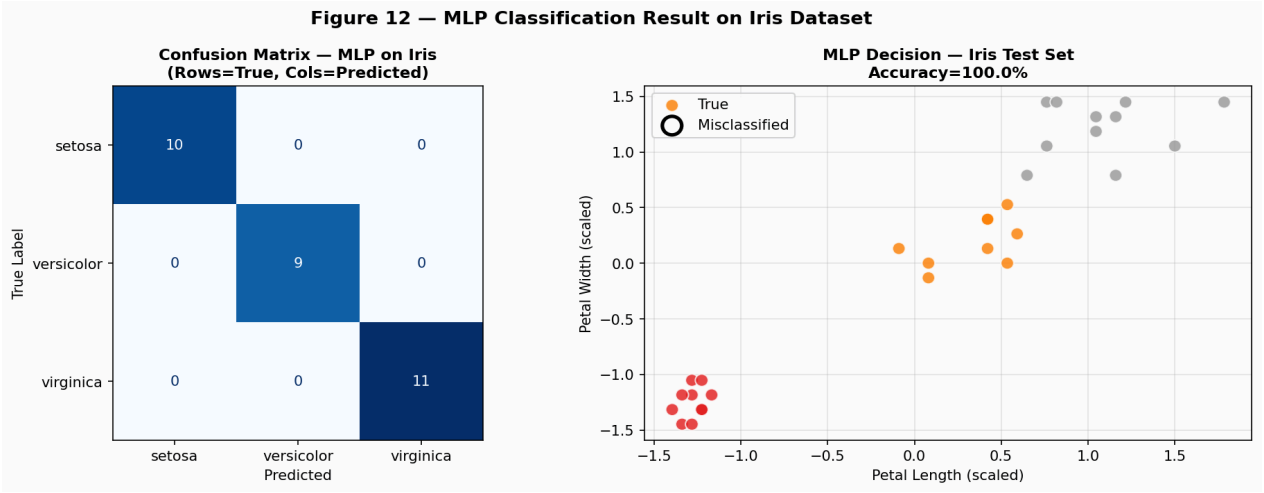


Figure 12 — MLP classification result on Iris (sklearn). Left: confusion matrix. Right: decision boundary with misclassified samples circled.

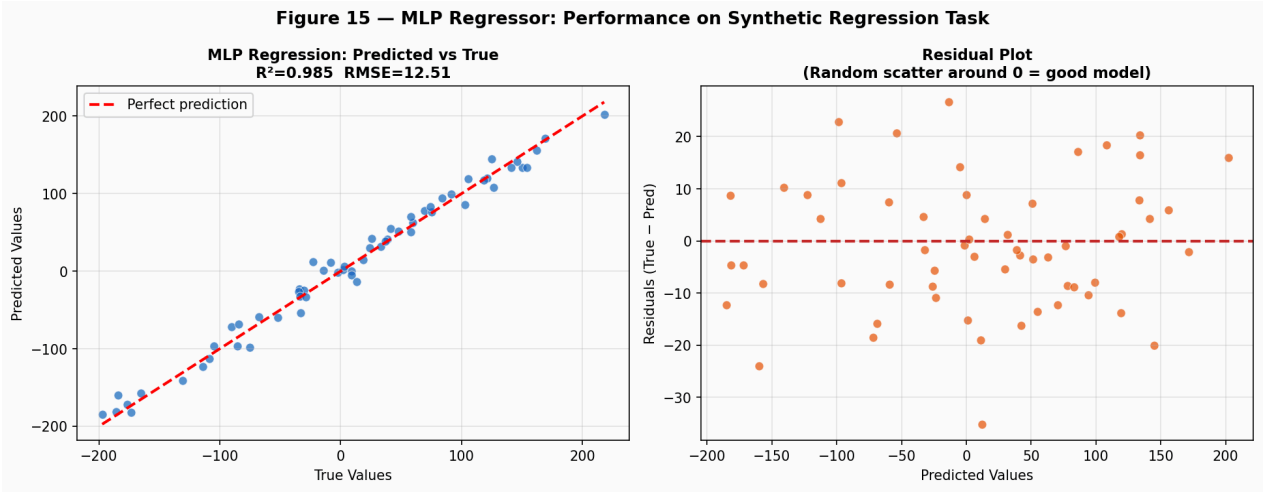


Figure 15 — MLP Regressor performance: predicted vs true (left) and residual plot (right).

9.1 Classification with MLPClassifier

```
python
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
```

```
# ── Dataset: Digits (1797 samples, 64 features, 10 classes) ──────
digits      = load_digits()
X, y        = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# ── Always scale ──────────────────────────────────────────────────
sc          = StandardScaler()
X_train_s   = sc.fit_transform(X_train)
X_test_s    = sc.transform(X_test)

# ── Build and train ───────────────────────────────────────────────
mlp = MLPClassifier(
    hidden_layer_sizes = (256, 128, 64),  # 3 hidden layers: 256, 128, 64
neurons
    activation         = 'relu',          # ReLU in hidden layers
    solver             = 'adam',          # Adam optimizer
    learning_rate_init = 0.001,           # Starting learning rate
    max_iter           = 500,             # Max epochs
    batch_size         = 64,              # Mini-batch size
    early_stopping     = True,            # Stop if val doesn't improve
    validation_fraction= 0.1,            # 10% for validation
    n_iter_no_change   = 20,              # Patience for early stopping
    random_state       = 42,
    verbose            = True
)

mlp.fit(X_train_s, y_train)

# ── Evaluate ──────────────────────────────────────────────────────
test_acc = mlp.score(X_test_s, y_test)
print(f"Test Accuracy: {test_acc:.4f}")

y_pred = mlp.predict(X_test_s)
print(classification_report(y_test, y_pred))

# ── Cross-validation for robust estimate ──────────────────────────
X_s    = sc.fit_transform(X)
cv_scores = cross_val_score(mlp, X_s, y, cv=5, scoring='accuracy')
print(f"5-fold CV: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")

# ── Loss curve ────────────────────────────────────────────────────
import matplotlib.pyplot as plt
plt.plot(mlp.loss_curve_); plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.title('Training Loss Curve'); plt.show()
```

9.2 Regression with MLPRegressor

```
• • •   python

from sklearn.neural_network import MLPRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error
import numpy as np

housing = fetch_california_housing()
```

```
X, y    = housing.data, housing.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
sc      = StandardScaler()
X_train_s = sc.fit_transform(X_train)
X_test_s  = sc.transform(X_test)

mlp_reg = MLPRegressor(
    hidden_layer_sizes = (128, 64, 32),   # Pyramid architecture
    activation         = 'relu',
    solver             = 'adam',
    learning_rate_init = 0.001,
    max_iter           = 300,
    early_stopping     = True,
    random_state       = 42
)

mlp_reg.fit(X_train_s, y_train)
y_pred = mlp_reg.predict(X_test_s)

print(f"R² Score : {r2_score(y_test, y_pred):.4f}")
print(f"MAE      : {mean_absolute_error(y_test, y_pred):.4f}")
print(f"RMSE     : {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
```

9.3 MLPClassifier Key Parameters

Parameter	Default	What It Controls	Good Starting Values
hidden_layer_sizes	(100,)	Architecture — tuple of neurons per layer	(64,32) or (128,64,32)
activation	relu	Hidden layer activation function	relu (default), tanh
solver	adam	Optimizer	adam for most cases
alpha	0.0001	L2 regularisation strength	0.0001 to 0.01
learning_rate_init	0.001	Initial learning rate	0.001 (Adam default)
max_iter	200	Maximum training epochs	200–500
batch_size	min(200, n_samples)	Mini-batch size	32, 64, 128
early_stopping	False	Stop if val_loss stagnates	Set to True
validation_fraction	0.1	Fraction of train data for validation	0.1–0.2

10 ANN vs Traditional ML Models

How neural networks differ from classical algorithms and when each wins.

Understanding **when to use neural networks vs traditional ML** is just as important as knowing how to build them. Neural networks are not always the right tool — and sometimes a simpler algorithm will beat a deep network on your specific problem.

10.1 The Core Difference

The Key Distinction in One Sentence

Traditional ML: You design features → algorithm finds the best combination of those features.
Deep Learning: You feed raw data → the network discovers and combines features itself.

This means: Deep Learning requires more data to shine, but removes the need for manual feature engineering — and scales better with data volume.

10.2 Head-to-Head Comparison

Property	Traditional ML(SVM, RF, XGBoost)	Artificial Neural Network
Feature engineering	Manual — you design features	Automatic — network learns features
Data requirements	Works well with 100–10K samples	Needs 10K+ for complex tasks; shines at 100K+
Training time	Seconds to minutes on CPU	Minutes to days, usually needs GPU
Interpretability	High (decision trees, coef)	Low — "black box" (except with SHAP/LIME)
Tabular data (small)	SVM/RF usually wins	Might underperform on < 5K rows
Images	SVM with HOG — limited	CNN completely dominates
Text/NLP	TF-IDF + LogReg — good baseline	Transformers (BERT, GPT) dominate
Hyperparameter tuning	Fewer, more intuitive	Many (LR, layers, neurons, dropout...)
Deployment	Easy (single model file)	Larger, needs TF/PyTorch runtime
Uncertainty estimates	Available (calibrated RF)	Requires extra work (MC Dropout, etc.)

10.3 Decision Guide — What to Use When

Situation	Best Choice	Why
Structured tabular, < 10K rows	XGBoost / Random Forest	Faster, more accurate, less tuning needed
Structured tabular, 10K–100K rows	Try both; Neural Net often competitive	Size starts to favour NN

Structured tabular, > 100K rows	Neural Network (TabNet, MLP)	NN learns complex interactions better at scale
Image classification	CNN (VGG, ResNet)	CNNs are designed for spatial pattern learning
Any text / NLP task	Transformer (BERT, GPT fine-tune)	Transformers dominate NLP by large margins
Time series forecasting	LSTM / Transformer / Prophet	Sequence models capture temporal dependencies
Anomaly detection (tabular)	Isolation Forest / Autoencoder	Both work; Autoencoder for more complex patterns
Need fast results & explainability	Random Forest / XGBoost	Quick, interpretable, often good enough

10.4 Why Neural Networks Win at Scale

The graph below (conceptual) illustrates a fundamental property: Traditional ML algorithms tend to **plateau in performance** as data grows — adding more data yields diminishing returns. Neural networks, by contrast, continue to **improve with more data**. This "scaling law" is why large language models with trillions of parameters trained on the entire internet outperform everything on language tasks — and why deep learning has taken over domains with abundant data (images, text, audio).

Data Volume	Best Traditional ML Accuracy	Best Neural Net Accuracy	Winner
100–1K samples	~82% (RF)	~75% (underfits)	Traditional ML
1K–10K samples	~88% (XGBoost)	~87% (similar)	Draw
10K–100K samples	~91%	~93%	Neural Network
100K–1M samples	~92% (plateaus)	~96%	Neural Network
> 10M samples	~92% (still plateaus)	~99%+	Neural Network by far

11 Designing a Neural Network — Complete Checklist

The full workflow from raw data to a working model.

11.1 Step-by-Step Design Process

1. **Define the problem:** Classification or Regression? Binary, multi-class, or multi-label? What metric matters?
2. **Collect & explore data:** How many samples? How many features? Any class imbalance? Missing values?
3. **Preprocess:** Scale features. Encode categories. Handle missing values. Train/test split (80/20 or 70/30).
4. **Design input layer:** Size = number of features (after encoding/flattening).
5. **Design hidden layers:** Start with 2 layers. Pyramid (256→128) or constant (128→128). Use ReLU.
6. **Design output layer:** Size and activation based on task (see Chapter 2 table).
7. **Choose loss function:** Matches output layer. Binary CE, Categorical CE, MSE.
8. **Choose optimizer:** Adam with lr=0.001 for almost everything.
9. **Add regularisation:** Dropout (0.2–0.3) + EarlyStopping + BatchNorm.
10. **Train and monitor:** Watch loss curve + train vs val accuracy gap.
11. **Evaluate:** Use Accuracy/F1/AUC for classification; R^2/MAE/RMSE for regression.
12. **Tune if needed:** More/fewer layers, larger/smaller width, different LR, more dropout.

11.2 Architecture Selection Quick Reference

Task	Input Layer	Hidden Layers	Output Layer	Loss	Metric
Binary Classification	n features	Dense(64)+ReLUDense(32)+ReLU	Dense(1)Sigmoid	binary_crossentropy	Accuracy, AUC
Multi-Class (K classes)	n features	Dense(128)+ReLUDense(64)+ReLU	Dense(K)Softmax	sparse_categorical_CE	Accuracy, F1
Multi-Label (K labels)	n features	Dense(128)+ReLUDense(64)+ReLU	Dense(K)Sigmoid	binary_crossentropy (each)	F1, Hamming
Regression (1 output)	n features	Dense(128)+ReLUDense(64)+ReLU	Dense(1)Linear	mse or mae	R^2, MAE, RMSE
Regression (K outputs)	n features	Dense(128)+ReLUDense(64)+ReLU	Dense(K)Linear	mse	R^2 per output

11.3 Common Mistakes & Fixes

Mistake	Symptom	Fix
Not scaling inputs	Training very slow; loss oscillates wildly	Always apply StandardScaler before training
Too many layers for small data	Perfect train acc, terrible test acc	Reduce depth; add dropout; get more data
Learning rate too high	Loss explodes (NaN or huge values)	Reduce LR by 10x (e.g., 0.01 → 0.001)
Learning rate too low	Loss barely moves after 100 epochs	Increase LR or use Adam (not SGD)
Wrong output activation	Predictions outside [0,1] for classification	Binary → sigmoid; Multi-class → softmax
Wrong loss for task	Model trains but accuracy stays ~33% for 3 classes	Ensure loss matches task (CE for classification)
No validation set	Can't detect overfitting	Always set validation_split=0.2
Forgetting to scale test data	Test accuracy much worse than validation	Use scaler fitted on train only; transform both
Using accuracy with imbalance	97% accuracy on 97%/3% split means nothing	Use F1, AUC, Precision-Recall instead

Summary — Deep Learning Core Concepts at a Glance

Concept	One-Line Summary
Perceptron	Single artificial neuron: $z = \Sigma(x_i w_i)+b$, output = f(z). Cannot solve non-linear problems alone.
MLP	Stack of perceptrons in layers. Hidden layers learn representations. Can approximate any function.
Input Layer	Pass-through for features. Size = number of features. Always scale inputs.
Hidden Layer	Computation: linear transform + activation. Stack 2–4 for most tasks; use ReLU.
Output Layer	Size and activation decided by task (sigmoid/softmax/linear). Critical design choice.
Weights & Bias	Learnable parameters. Weights: how much each input matters. Bias: shifts activation.
Activation (ReLU)	Adds non-linearity. ReLU = default for hidden layers. Allows learning complex boundaries.
Softmax	Converts logits to probabilities summing to 1. Output layer for multi-class classification.
Loss Function	Measures how wrong the model is. MSE for regression; Cross-Entropy for classification.
Forward Pass	Data flows input→output; predictions and loss computed.
Backward Pass	Gradients computed via chain rule; flows output→input.
Gradient Descent	Update weights in direction that reduces loss: $w = w - \alpha \times \partial L/\partial w$
Mini-batch (32-64)	Default GD variant: balance of speed and gradient accuracy.
Adam Optimizer	Adaptive learning rate + momentum. Default optimizer (lr=0.001).
Dropout	Regularisation: randomly zero neurons during training. Prevents overfitting.
EarlyStopping	Stop training when val_loss stops improving. Prevents overfitting.
Overfitting	Low train loss, high val loss. Fix: more data, dropout, regularisation.
Computation Graph	Directed graph of all operations. Enables automatic backpropagation.